

Technical Documentation – Milestone 1

AI Smart Bug Analyzer and Fix Advisor (BugSense AI)

Prepared By: Harshitha Bejjipuram
Internship : Springboard Internship

1. Introduction

Software development projects often generate thousands of bug reports during the development and maintenance phases. Many of these reports describe issues that have already been encountered and resolved in the past. Since developers manually search previous issues, duplicate bug reports consume significant development time and reduce productivity.

The **AI Smart Bug Analyzer and Fix Advisor (BugSense AI)** is designed to solve this challenge by introducing an intelligent **Retrieval-Augmented Generation (RAG)** based bug analysis platform. Instead of relying only on keyword matching, the system converts bug reports into semantic vector embeddings and searches historical bug repositories to retrieve the most relevant previously resolved issues.

Milestone 1 establishes the core infrastructure of the platform by implementing document ingestion, vector embedding generation, semantic similarity search, and duplicate bug detection. This milestone serves as the foundation for the AI-powered diagnostic agents introduced in the next milestone.

2. Problem Statement

In large software projects, developers frequently encounter the following problems:

- Duplicate bug reports increase workload.
- Manual searching of historical bugs consumes valuable time.
- Traditional keyword-based search often returns irrelevant results.
- Debugging similar issues repeatedly delays software releases.
- Knowledge from previously resolved bugs is underutilized.

The objective of this milestone is to build an intelligent retrieval system capable of understanding the meaning of bug reports rather than simply matching keywords.

3. Objectives

- The primary objectives of Milestone 1 are:
- Design an enterprise-ready bug ingestion pipeline.
- Support multiple document formats such as LOG, TXT, PDF, DOCX, and CSV.

- Convert bug reports into semantic vector embedding
- Store embeddings efficiently using Facebook AI Similarity Search (FAISS).
- Retrieve historical bugs using semantic similarity.
- Automatically detect duplicate bug reports.
- Provide REST APIs for seamless frontend integration.
- Develop a responsive React dashboard for bug submission and monitoring.

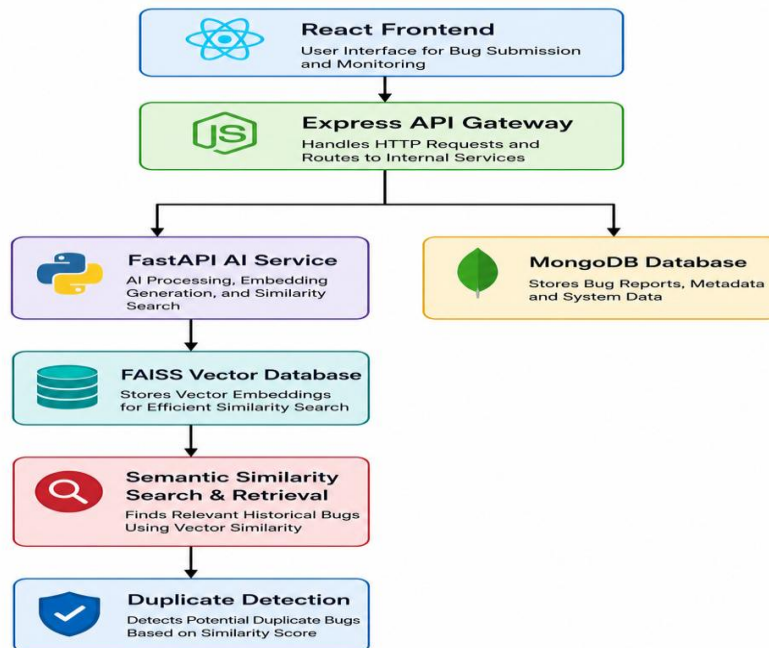
4. Project Overview

BugSense AI follows a modular architecture that separates the frontend, backend gateway, AI microservice, and vector database.

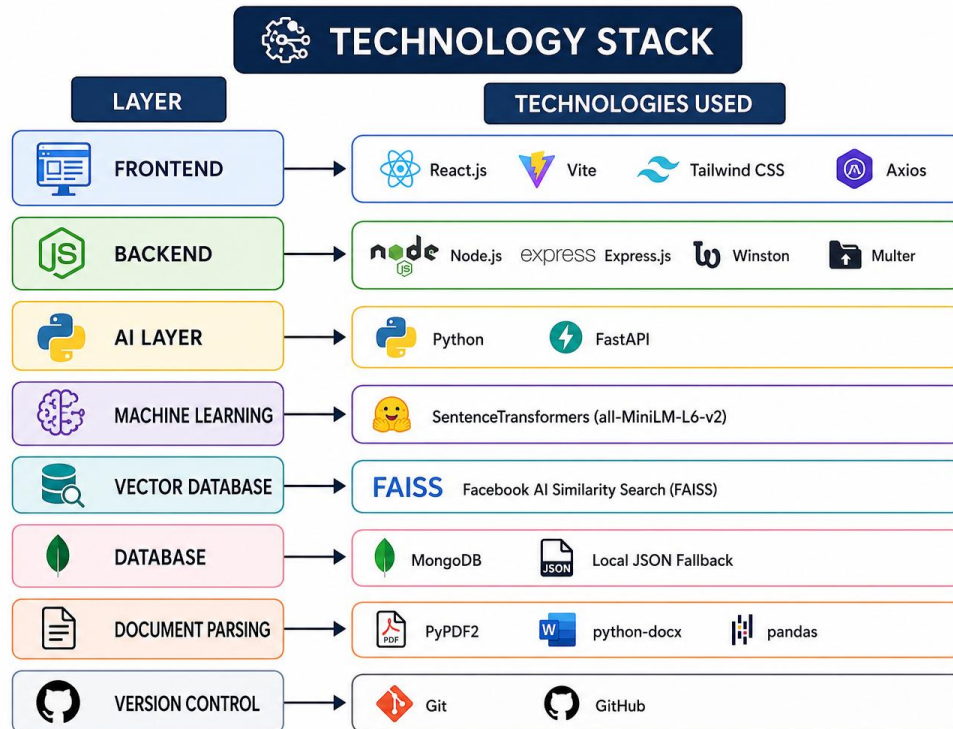
The overall workflow begins with a user submitting a bug report through the React dashboard. The request is forwarded to the Express.js API Gateway, which communicates with the FastAPI AI service. The AI service extracts textual information, generates semantic embeddings, performs similarity search using FAISS, and returns the most relevant historical bug reports.

If a highly similar issue is found, the system labels it as a Potential Duplicate Bug, thereby reducing redundant issue creation.

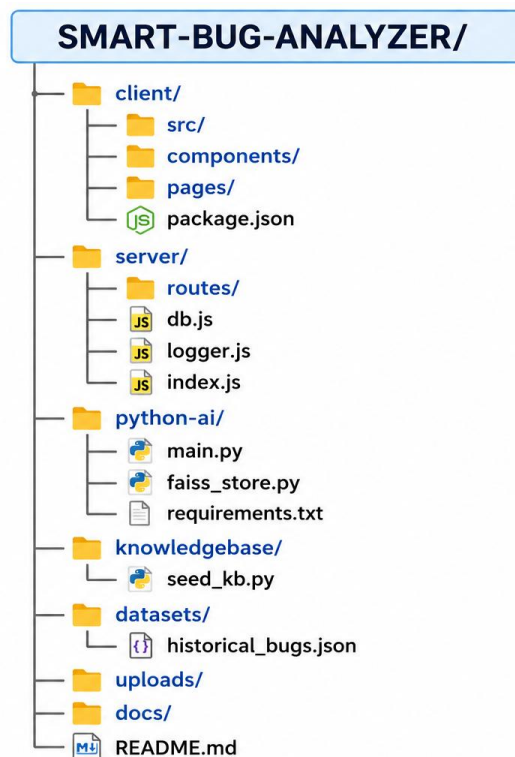
5. System Architecture



6. Technology Stack



7. Project Directory Structure



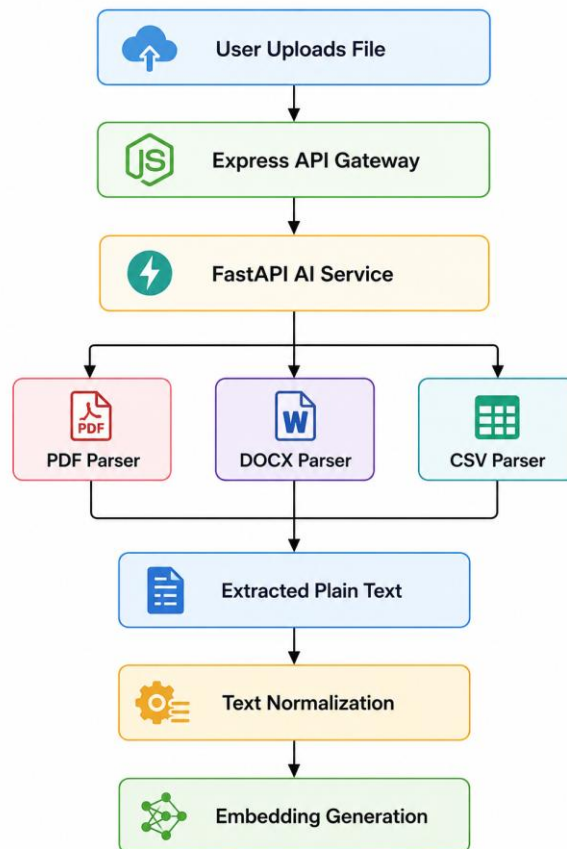
8. Document Ingestion Pipeline

File Type	Library Used	Purpose
.log / .txt	Python File Reader	Runtime log processing
.pdf	PyPDF2	PDF text extraction
.docx	python-docx	Microsoft Word document parsing
.csv	Pandas	CSV table extraction

This flexible ingestion mechanism enables developers to upload bug reports in different formats without requiring manual preprocessing.

9. Document Processing Workflow

The following workflow illustrates the complete processing pipeline implemented in Milestone 1.



10. Semantic Embedding Generation

After extracting textual information, the system converts each bug report into a high-dimensional semantic vector.

The project uses the following embedding model:

Sentence Transformer

all-MiniLM-L6-v2

This transformer converts natural language bug descriptions into **384-dimensional dense vectors**.

Unlike traditional keyword matching, semantic embeddings capture the contextual meaning of bug reports. Therefore, even if two bug reports use different wording, the system can still identify them as similar if they describe the same underlying issue.

Advantages of Semantic Embeddings

- Captures contextual meaning
- Language-independent semantic comparison
- Faster retrieval
- Better duplicate detection
- High retrieval accuracy

11. FAISS Vector Database

To perform efficient similarity search, Milestone 1 integrates **Facebook AI Similarity Search (FAISS)**.

FAISS stores all embedding vectors inside a searchable vector index.

The project uses the following index type:

IndexFlatIP

This index computes cosine similarity between the query vector and stored vectors.

Why FAISS?

- Extremely fast vector search

- Low memory consumption
- Suitable for large datasets
- Production-ready implementation
- Supports semantic retrieval

The generated vector index is stored locally as:

embeddings/faiss_index.bin

Metadata associated with every vector is stored separately to map vectors back to historical bug records.

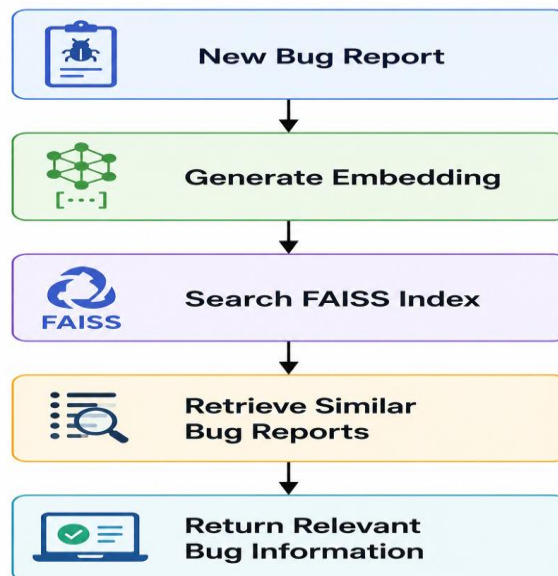
12.Retrieval-Augmented Generation (RAG) Pipeline

Milestone 1 implements a Retrieval-Augmented Generation (RAG) architecture.

Instead of generating responses solely from an AI model, the system first retrieves relevant historical bug reports from the knowledge base.

RAG Workflow

This retrieval-first approach improves the quality of bug analysis by providing historical context before any further AI-based reasoning



13. Semantic Similarity Search

Whenever a new bug report is submitted, the following information is combined:

- Bug Title
- Bug Description
- Stack Trace

The combined text is converted into a semantic vector.

The FAISS engine compares this vector with all previously indexed bug vectors.

The similarity score is calculated using cosine similarity, allowing the system to identify semantically related bug reports even when different words are used.

This process significantly improves retrieval accuracy compared to conventional keyword-based search.

14. Duplicate Bug Detection

Duplicate bug reports waste developer time and create unnecessary maintenance overhead.

To address this problem, BugSense AI automatically checks every incoming bug report against the historical knowledge base.

The duplicate detection rule is:

Similarity Score > 90%

If this threshold is exceeded, the bug is marked as:

Potential Duplicate Bug

The system immediately returns:

- Matching Bug ID
- Similarity Score
- Historical Bug Information

This feature reduces redundant issue creation and accelerates the debugging process.

15. Database Management

Milestone 1 provides a resilient database architecture.

Primary Database

- MongoDB

Backup Storage

- Local JSON Database

If MongoDB is unavailable due to connection issues or server downtime, the Express server automatically switches to the local JSON database. This failover mechanism ensures uninterrupted system operation and prevents data loss during development.

16. REST API Endpoints

The BugSense AI platform exposes multiple RESTful APIs through the Express.js gateway to enable communication between the frontend, backend, and AI microservice.

The APIs implemented in Milestone 1 are summarized below.

API Endpoint	HTTP	Description
/api/upload	POST	Uploads documents and extracts text from supported file formats.
/api/submitBug	POST	Creates a new bug report, generates embeddings, performs duplicate detection and stores the bug.
/api/bugs	GET	Retrieves all submitted bug reports from the database.
/api/similar/:id	GET	Returns semantically similar historical bugs from the FAISS index.

17. React Dashboard Features

The frontend dashboard provides an interactive interface for developers to submit bug reports and review previously reported issues.

Dashboard Modules

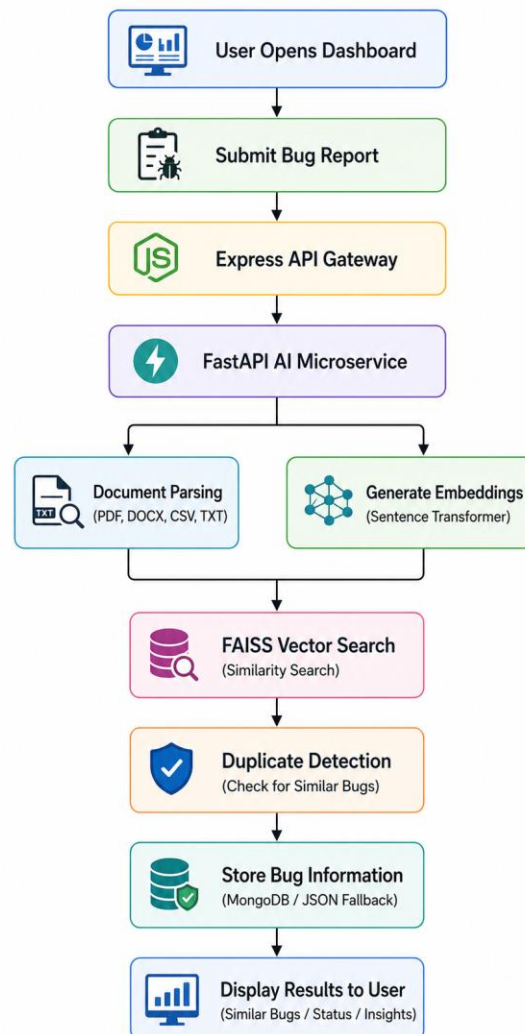
- Dashboard Overview

- Submit Bug Form
- Knowledge Base
- Similar Bug Viewer
- Bug Statistics
- Theme Toggle (Dark / Light Mode)

The dashboard is built using **React.js**, **Vite**, and **Tailwind CSS**, ensuring a responsive and user-friendly experience across different screen sizes.

18. End-to-End System Workflow

The complete execution flow of Milestone 1 is illustrated below.



19. Testing and Validation

Several functional tests were conducted to verify the correctness of the implemented modules.

TESTING AND VALIDATION RESULTS		
 Module	 Status	
 Document Upload	 Passed	
 PDF Parsing	 Passed	
 DOCX Parsing	 Passed	
 CSV Parsing	 Passed	
 Embedding Generation	 Passed	
 FAISS Search	 Passed	
 Duplicate Detection	 Passed	
 REST APIs	 Passed	
 Dashboard Integration	 Passed	

Performance Validation

The platform was evaluated using multiple historical bug reports.

The evaluation demonstrated:

- Successful document parsing.
- Accurate semantic retrieval.
- Fast vector similarity search.
- Reliable duplicate detection.

- Stable communication between frontend and backend services.

20. Key Achievements

Milestone 1 successfully established the core infrastructure of BugSense AI.

Major achievements include:

- Multi-format document ingestion.
- Automatic text extraction.
- Semantic embedding generation using Sentence Transformers.
- High-speed similarity retrieval using FAISS.
- Duplicate bug detection based on semantic similarity.
- MongoDB integration with local JSON fallback.
- REST API implementation.
- Interactive React dashboard.
- Modular project architecture.

21. Advantages of the Implemented Solution

Compared with traditional keyword-based bug tracking systems, BugSense AI provides several advantages.

- Faster bug retrieval.
- Reduced duplicate issue reporting.
- Better utilization of historical bug knowledge.
- Semantic understanding instead of keyword matching.
- Scalable architecture for enterprise applications.
- Easy integration with AI-based diagnostic agents.

22. Challenges Faced

During the implementation of Milestone 1, several technical challenges were encountered.

- Handling multiple document formats.
- Maintaining consistent text extraction quality.
- Efficient storage of embedding vectors.
- Integrating Python AI services with the Node.js backend.
- Implementing fallback mechanisms for database failures.
- Optimizing semantic similarity search performance.

These challenges were addressed through modular architecture, reusable components, and efficient vector indexing.

23. Future Enhancements

Milestone 1 serves as the foundation for several advanced AI capabilities.

Future enhancements include:

- Multi-Agent Bug Diagnosis.
- Root Cause Analysis Agent.
- AI-powered Fix Recommendation.
- Automatic Code Suggestion.
- Cloud-based Vector Database.
- CI/CD Integration.
- Real-time Monitoring Dashboard.
- User Authentication and Role Management.
- Bug Priority Prediction.
- Automated Bug Report Summarization.

These features will be implemented in the subsequent milestones to enhance the platform's intelligence and scalability.

24. Conclusion

Milestone 1 successfully establishes the foundational architecture of the **AI Smart Bug Analyzer and Fix Advisor (BugSense AI)** platform. The implementation integrates

document ingestion, semantic embedding generation, FAISS-based vector search, and duplicate bug detection into a unified workflow.

By leveraging Retrieval-Augmented Generation (RAG), the platform enables developers to retrieve historically similar bug reports efficiently, reducing duplicate issue creation and accelerating debugging. The modular architecture, scalable design, and seamless integration of frontend, backend, and AI services provide a strong base for the advanced multi-agent diagnostic capabilities implemented in Milestone 2.